

Estructuras de datos

Modo de cursada

- Tema nuevo cada semana
- Fin de semana previo habilitamos pestaña en campus con video y ejercicios
- Lunes: Clase teórico-práctica presencial
- Viernes: Clase práctica virtual sincrónica
- Consultas asincrónicas por Discord en la semana (**#matricularse-materias-discord**)

Temas

- Primer Parcial:
 - Tipos de datos abstractos
 - Recursividad
 - Arreglos uni y multidimensionales
- Segundo Parcial:
 - Pilas y Colas dinámicas
 - Diccionarios y conjuntos
 - Listas enlazadas
 - Arboles

Fechas importantes

- Cuatrimestre: 10/08 al 27/11
- Primer parcial: 21 de Septiembre.
- Segundo Parcial: 16 de Noviembre.
- Recuperatorios: 20 y 27 de Noviembre.

Regimen académico

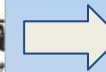
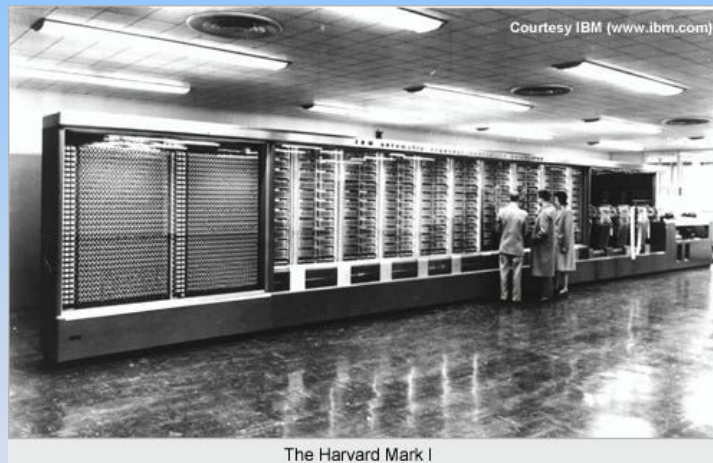
- Nuevo sistema de inscripción / asistencia
- Parciales se aprueban con 4.
- Promoción
 - Promedio mayor o igual a 7
 - Más de 6 en ambos parciales o recuperatorios.
- Regularización
 - Promedio entre 4 y 6
 - Integrador
 - Final

Introducción a Python

Parte 1

Procesamiento
Algoritmo (programa)

Entrada



Salida

Que es un algoritmo?

Conjunto finito de pasos/operaciones para resolver un problema

Que es un lenguaje de programación?

Conjunto de reglas y símbolos que permiten escribir instrucciones para que una computadora ejecute tareas.

Que es un programa?

Implementación de un algoritmo en un archivo de texto
Usando un lenguaje de programación
“Ejecuto” el programa en la computadora

Donde ejecutamos un programa?

Si no tiene interfaz gráfica usamos la consola o línea de comandos (Windows CMD / Linux SHELL)

Que es PYTHON?

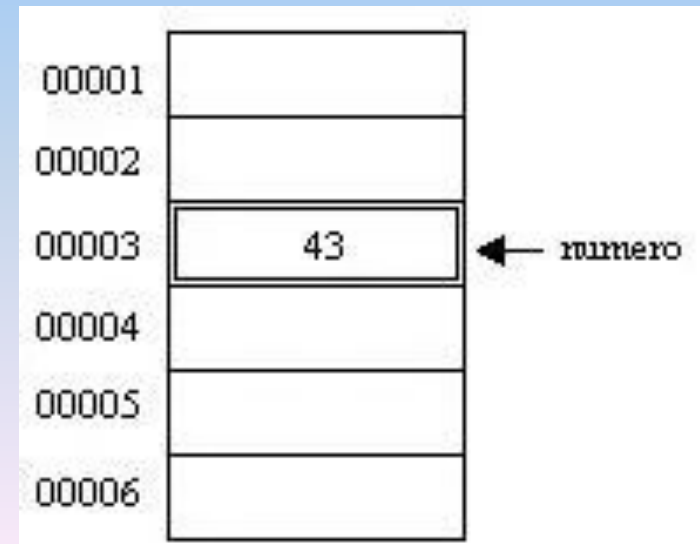
- Python es un lenguaje de programación interpretado con una sintaxis que favorece la legibilidad del código.
- Creado a finales de los 80s por Guido Van Rossum
- Multiparadigma: Orientado a objetos, programación funcional y programación imperativa.
- Licencia GNU GPL
- No se compila

Elementos básicos de lenguajes de programación

- Variables
- Palabras reservadas
- Operadores
- Expresiones
- Entrada / Salida
- Comentarios
- Estructura de programa

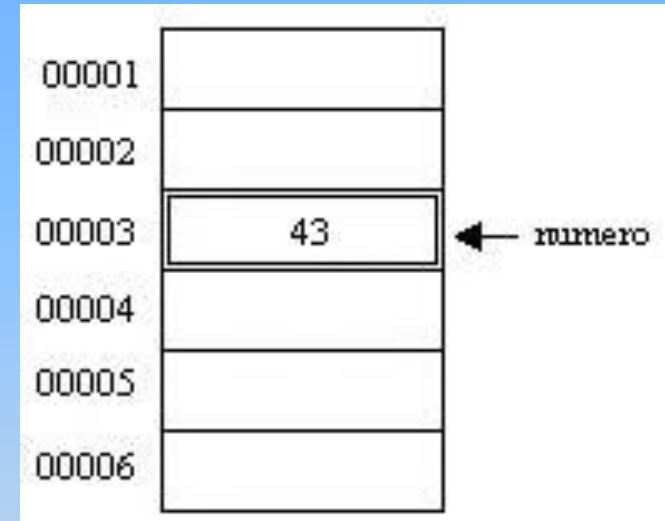
Variables

- Contenedores de datos
- Ocupan espacio en RAM
- Nombre / Identificador (posición)
- Tipo (espacio ocupado y codificación)



Variables

- Nombre / Identificador
 - Alias para la posición
 - Formado por letras, números y caracteres especiales
 - No puede iniciar por un número
- Tipo
 - Define espacio de memoria ocupado
 - Define codificación (interpretación de 0s y 1s en la memoria)



Tipos de datos primitivos

- Implementados en el lenguaje
- Representación en la computadora de datos numéricos, lógicos, caracteres (símbolos), etc.

Númericos

Enteros:

- Tipo “int”
- Números enteros con signo

Punto flotante

- Tipo “float”
- Números reales con decimales
- Precisión: Cantidad de cifras decimales

Cadenas de caracteres (Strings)

- Tipo “str”
- Serie de símbolos
 - Letras, números y especiales (ej. espacios)
 - Palabras
 - Se definen entre comillas (dobles o simples)

Booleanos

- Tipo “bool”
- Lógica booleana
 - Verdadero (True)
 - Falso (False)

Declaración de variables

- Las variables no se declaran definiendo un tipo
- El tipado es automático y dinámico
- En general NO se pueden realizar operaciones entre variables de distinto tipo
- Cambio de tipo de variable (*casteo*):
 - `int()`
 - `float()`
 - `str()`

Declaración de variables

```
nombreVariable = valor
```

Ejemplos:

```
dias = 2  
decision = True  
letra = "C"  
radio = 25.63  
dias = "lunes"
```

```
x , y , z = 34 , 25 , 12  
x = y = "Hola"
```



Palabras reservadas

```
help> keywords
```

```
Here is a list of the Python keywords. Enter any keyword to get more help.
```

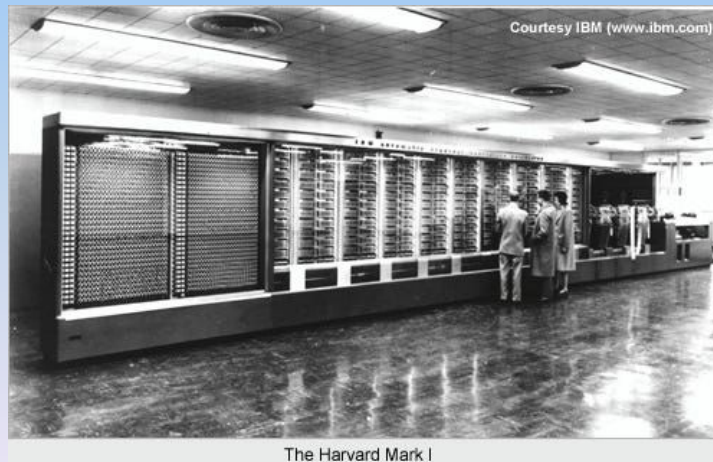
False	break	for	not
None	class	from	or
True	continue	global	pass
__peg_parser__	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield

```
help> █
```

Entrada / Salida

Procesamiento
Algoritmo (programa)

Entrada



Salida

Salida

- Por el monitor (“imprimimos” en la consola)
- Archivos

Función "print"

```
print(<constante>)
```

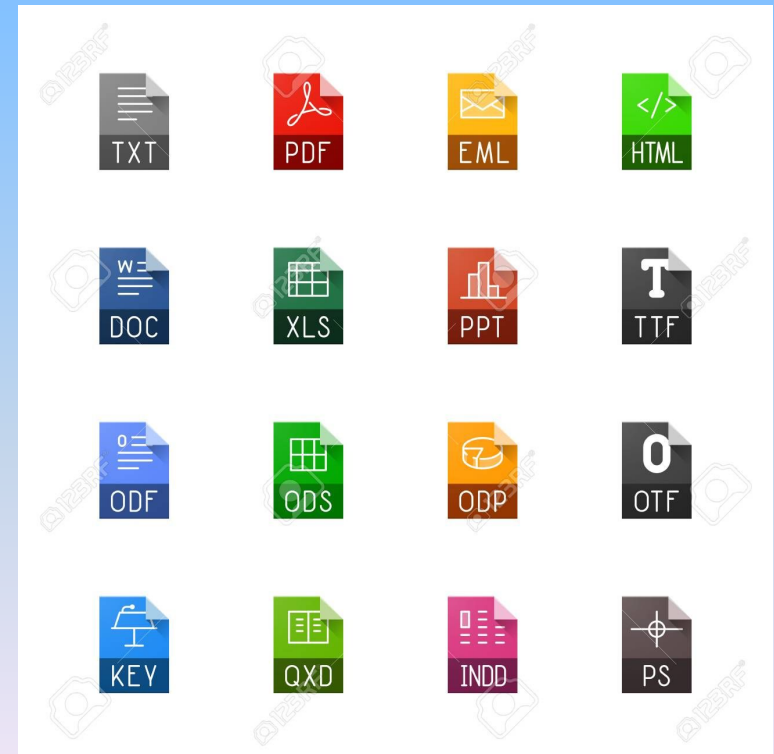
```
print("texto a imprimir")
```

```
print(variable) -> Imprime contenido de la variable
```



Entrada

- Desde el teclado
- Archivos



Entrada

- Para leer los datos que se introducen en el teclado se utiliza la funcion **input()**
- Se puede utilizar de la siguiente manera:

```
variable = input("Ingrese un numero: ")
```

Entrada

- Por defecto, “input” convierte los datos ingresados a tipo “str”
- Pero es posible ‘castear’ las variables

```
variable = int(input("Ingrese un numero entero: "))
```

Operadores

- Permiten realizar operaciones / calculos entre variables y / o constantes
- Tipos:
 - Asignación
 - Aritméticos
 - Comparación
 - Lógicos

Operadores de asignación

- Asignar valores en variables

– =

– += / -=

Operadores aritméticos

operador	significado
+	Suma
-	Resta
*	Producto
/	División
//	División entera
%	Módulo (resto)

- Tipo de entrada: Cualquiera (depende del operador)
- Tipo de salida: Depende del tipo de dato de entrada
- Diferente función con diferentes tipos de datos



Operadores de comparacion

operador	significado
<	Menor
>	Mayor
>=	Mayor o igual
<=	Menor o igual
==	Igual
!=	Distinto

- Tipo de entrada: Cualquiera (comparables entre si)
- Tipo de salida: bool

Operadores lógicos

operador	significado
not	No lógico (NOT)
and	“Y” lógico (AND)
or	“O” lógico (OR)

- Tipo de entrada: bool
- Tipo de salida: bool

Expresiones

- Resultado booleano
- Condiciones complejas, no alcanza con solo un operador
- Formadas por un conjunto de variables, constantes, operadores aritméticos, operadores de comparación, operadores lógicos, etc

var1 + var2 > 20

Ejercicios

- Podemos hacer los ejercicios de la primer parte

Estructuras de control

- Ya sabemos declarar variables, hacer operaciones de distintos tipos entre ellas, ingresar y sacar datos del programa
- Nos falta estructurar el código
- Concepto de “Bloque de código”
- Indentación obligatoria

Estructuras de control

- Permiten modificar el orden de ejecución de las líneas del programa
- Ejecución condicional o repetitiva de líneas de código según resultado de expresiones
- Tipos:
 - Selectivas
 - Repetitivas

Estructuras de control selectivas

- Bifurcaciones en el flujo del programa.
- Decidir que hacer a partir de evaluar una condición.
- Uso de operadores y expresiones lógicas.

Alternativas simples

- Si – Entonces / If – then.
- Ejecuta una acción sólo cuando se cumple una condición

```
if condicion:  
    S1  
    S2  
    ...  
    Sn
```

Alternativas dobles

- Si – Entonces – si_no / If – then – else.

```
if condicion:  
    acciones S1  
else:  
    acciones S2
```

Alternativas múltiples

- Si – Entonces – si_no – entonces ...
- If – then – else if – then ...

```
if condicion1:  
    acciones S1  
elif condicion2:  
    acciones S2  
elif condicion3:  
    acciones S3
```

Estructuras de control repetitivas

- Conjunto de operaciones que se deben repetir varias veces.
- Ciclo o bucle: Parte de un programa que se repite (iteración) un número dado de veces o mientras se cumpla una condición.
- Ciudadano con bucles infinitos.

Estructuras de control repetitivas

- Desde – Hasta / For

```
for variable in elemento_iterable:  
    <acciones>
```

Estructuras de control repetitivas

- Desde – Hasta / For
- Uso alternativo a iterar colecciones:
 - Ejemplo: Repetición de bloque de código 10 veces usando la función range:

```
for var in range(10):  
    <bloque codigo>
```

Estructuras de control repetitivas

- Mientras / While

```
while condicion:  
    <accion>
```

Comentarios

- Líneas de código que la computadora no “ejecuta”
- Guardar información
- Si son de una línea debe comenzar con "#" y si ocupan más de una línea van entre ' ' '



Estructura de un programa

- Declaración de variables
- Sentencias que implementan el algoritmo

Identificador del programa o módulo

{sección de declaraciones}

inicio

{datos de entrada}

{sentencias imperativas, que ejecutan el algoritmo correspondiente}

{datos de salida}

fin

Algunas consideraciones finales sobre Python

- Hay diferencia entre mayúsculas y minúsculas.
- Las línea de código no deben terminar con ";" "
- El indentado es obligatorio, ya que no se usan llaves para separar bloques de codigo